Software Quality Assurance

HNDIT4022

Assignment

Name : W.B.M.N. Dasuni

Index No : DEHIT/2324/F/070

Designing and Evaluating Software Testing Strategies for a Student Management System

Task 1: Introduction to Software Testing

Definition of Software Testing

Software testing is the process of evaluating and verifying a software application to determine whether it meets specified requirements and functions correctly. The primary purpose of testing is to identify defects, errors, or missing requirements before the software is released to users.

According to software engineering principles, testing helps ensure that software products are reliable, secure, efficient, and capable of performing their intended functions.

Importance of Testing in Software Quality Assurance

Software testing is a fundamental activity within Software Quality Assurance (SQA). It **helps organizations deliver high-quality software products by ensuring that defects are identified and corrected before deployment.**

The importance of software testing:

1. Improves Software Quality - Testing ensures that the software behaves as expected and meets user requirements.

2. Detects Defects Early - Finding defects during development reduces the cost and effort required for fixing them later.

3. Enhances Security - Testing identifies vulnerabilities that could expose sensitive student information to unauthorized users.

4. Improves User Satisfaction - Users are more satisfied when software functions correctly without crashes or errors.

5. Reduces Maintenance Costs - Early detection of defects reduces future maintenance and support expenses.

6. Ensures Reliability - Testing helps guarantee stable and reliable system operation under different conditions.

Reasons for Software Failures

Software failures can occur due to several reasons:

1. Requirement Errors

Incomplete, incorrect, or misunderstood requirements may result in software that does not satisfy user expectations.

Example:
If the requirement states that students should be able to register for courses online, but developers misunderstand the process, the implemented functionality may not meet user needs.

2. Programming Errors

Developers may introduce coding mistakes during implementation.

Examples include:

- Incorrect calculations
- Logical errors
- Database query mistakes
- Input validation failures

3. Integration Problems

Different modules may work correctly independently but fail when integrated together.

Example:
The Student Registration Module may successfully save student information, but the Database Module may fail to retrieve the data properly.

4. Poor System Design

Weak architectural design may result in performance issues, security weaknesses, and scalability problems.

5. Environmental Issues

Software may fail due to server failures, network issues, hardware problems, or incompatible software versions.

Task 2: Testing Levels and Types

Testing Levels

Software testing is performed at different levels throughout the Software Development Life Cycle (SDLC).

1. **Unit Testing -** Unit Testing involves testing individual components or functions independently.

   Purpose:

- Verify that each unit works correctly.
- Detect coding errors early.

Example in Student Management System: Testing the function that calculates a student's GPA.

Advantages:

- Early bug detection.
- Easier debugging.
- Improved code quality.

2. Integration Testing - Integration Testing verifies interactions between combined modules.

Purpose:

- Ensure modules communicate correctly.
- Identify interface defects.

Example: Testing whether student registration information entered through the User Interface is correctly stored in the database.

Advantages:

- Detects communication errors.
- Ensures data consistency.

3. System Testing - System Testing evaluates the complete integrated system.

Purpose:

- Validate functional and non-functional requirements.
- Test the entire system in a realistic environment.

Example: Testing student registration, attendance, examination, and report generation modules together.

Advantages:

- Validates overall system behavior.
- Simulates real-world usage.

4. Acceptance Testing - Acceptance Testing is performed by end users or clients.

Purpose:

- Verify that the software satisfies business requirements.
- Determine readiness for deployment.

Example: School administrators test the system before implementation.

Advantages:

- Confirms user satisfaction.
- Reduces deployment risks.

Testing Types Relevant to Student Management System

1. Security Testing - Security Testing evaluates the system's ability to protect sensitive information.

Importance: Student records contain confidential information such as:

- Student names
- Contact information
- Examination results
- Financial records

Security testing verifies:

- Authentication mechanisms
- Authorization controls
- Password protection
- Data encryption

Expected Outcomes:

- Unauthorized access prevented.
- Student data protected.

2. Performance Testing - Performance Testing evaluates responsiveness, speed, stability, and scalability.

Importance: The Student Management System may serve hundreds or thousands of users simultaneously.

Performance testing measures:

- Response time
- Throughput
- Server performance

- Database performance

Expected Outcomes:

- Fast response times.
- Stable operation during peak usage.

Task 3: Test Case Design

Selected Module: Student Login Module

Test Design Technique: Boundary Value Analysis (BVA)

Assumptions:

Username Length:
Minimum = 3 characters
Maximum = 20 characters

Password Length:
Minimum = 6 characters
Maximum = 15 characters

Test case ID	Description	Input Data	Expected Output	Actual Output	Status
TC001	Username below minimum length	ab	Validation error	Validation error displayed	Pass
TC002	Username at minimum length	abc	Accepted	Accepted	Pass
TC003	Username above minimum length	abcd	Accepted	Accepted	Pass
TC004	Username at maximum length	20 characters	Accepted	Accepted	Pass
TC005	Username above maximum length	21 characters	Validation error	Validation error	Pass
TC006	Password below minimum length	12345	Validation error	Validation error	Pass

TC007	Password at minimum length	123456	Accepted	Accepted	Pass
TC008	Password at maximum length	15 characters	Accepted	Accepted	Pass
TC009	Password above maximum length	16 characters	Validation error	Validation error	Pass
TC010	Valid username and password	Student01/ Password123	Login successful	Loging successful	Pass

Task 4: White Box Testing vs Black Box Testing

White Box Testing - White Box Testing is a software testing technique that examines the internal structure, logic, and code of an application.

Characteristics:

- Requires programming knowledge.
- Tests internal code paths.
- Focuses on algorithms and logic.
- Usually performed by developers.

Advantages:

- Finds hidden coding errors.
- Improves code quality.
- Measures code coverage.

Disadvantages:

- Requires technical expertise.
- Time-consuming.

Black Box Testing - Black Box Testing evaluates software functionality without examining internal code.

Characteristics:

- Focuses on inputs and outputs.
- Tests requirements and functionality.
- Performed by testers and users.

Advantages:

- User-oriented testing.
- No programming knowledge required.
- Effective for validating requirements.

Disadvantages:

- Limited visibility into internal logic.
- Some hidden defects may remain undetected.

Comparison Between White Box and Black Box Testing

Feature	White Box Testing	Black Box Testing
Code knowledge	Required	Not required
Focus	Internal logic	External functionality
Performed by	Developers	Testers
Test basis	Source code	Requirements
Objective	Verify implementation	Verify functionality

Most Suitable Method for Login Module

Black Box Testing is more suitable for the Login Module because:

1. Login functionality is user-facing.
2. Requirements focus on user input and system output.
3. Internal code knowledge is unnecessary.
4. User experience and validation are primary concerns.

Task 5: Risk Analysis

Product Risks

Risk 1: Unauthorized Access

Description: Hackers may gain access to student records.

Likelihood: High
Impact: High

Risk 2: Data Loss

Description: Database failures may result in loss of student information.

Likelihood: Medium
Impact: High

Project Risks

Risk 3: Schedule Delay

Description: Development tasks may exceed planned timelines.

Likelihood: Medium
Impact: Medium

Risk 4: Lack of Skilled Testers

Description: Insufficient testing expertise may reduce testing effectiveness.

Likelihood: Low
Impact: Medium

Risk Matrix

Risk	Likelihood	Impact	Risk Level
Unauthorized access	High	High	Critical
Data loss	Medium	High	High
Schedule delay	Medium	Medium	Medium
Lack of skilled testers	Low	Medium	Low

Task 6: Test Automation Tool

Selected Tool: Selenium

Introduction

Selenium is a widely used open-source automation testing framework designed for testing web applications.

Features of Selenium

1. Cross-browser testing.
2. Multi-language support.
3. Parallel test execution.
4. Automated regression testing.

5. Integration with CI/CD tools.

Advantages

1. Free and open source.
2. Supports multiple browsers.
3. Large community support.
4. Reusable test scripts.
5. Faster execution of repetitive tests.

Limitations

1. Requires programming skills.
2. Limited support for desktop applications.
3. Test maintenance can be challenging.
4. Dynamic web elements may require complex handling.

Using Selenium in Student Management System

Selenium can automate:

- Login functionality testing.
- Student registration testing.
- Attendance management testing.
- Course enrollment testing.
- Result publication testing.

Benefits:

- Faster testing.
- Reduced manual effort.
- Improved accuracy.
- Supports regression testing.

Task 7: Conclusion

Software testing is an essential process for ensuring software quality, reliability, security, and performance. This report examined software testing strategies for a Student Management System. Different testing levels including Unit Testing, Integration Testing, System Testing, and Acceptance Testing were discussed. Security Testing and Performance Testing were identified as important testing types for protecting student information and maintaining system efficiency.

Boundary Value Analysis was applied to design test cases for the Login Module. White Box Testing and Black Box Testing were compared, and Black Box Testing was determined to be the most suitable approach for the selected module. Risk analysis identified major product and project risks, while Selenium was selected as an effective automation testing tool.

Overall, Quality Assurance plays a crucial role in software development by reducing defects, minimizing risks, improving customer satisfaction, and ensuring successful software deployment. Effective testing strategies contribute significantly to delivering high-quality software systems that meet user expectations and business requirements.

References

1. Pressman, R. S. (2020). Software Engineering: A Practitioner's Approach.
2. Sommerville, I. (2019). Software Engineering (10th Edition).
3. Myers, G. J., Sandler, C., & Badgett, T. (2018). The Art of Software Testing.
4. ISTQB Foundation Level Syllabus.
5. IEEE Software Testing Standards.

************END************